



Academic Year	Module	Assessment Number	Assessment Type
2026	Artificial Intelligence and Machine Learning	2	Report

Student Id : 2431866
Student Name : Sansar Dangol
Section : G16
Module Leader : Siman Giri
Tutor : Anish Khatiwada
Submitted on : 10-05-2026

Garbage Classification Using Convolutional Neural Networks and Transfer Learning

Abstract

The aim of this report is to explore the application of Convolutional Neural Networks (CNNs) and Transfer Learning to classify images of garbage. This study utilizes the Garbage Classification dataset from Kaggle that includes 15,515 images of waste types, such as plastic, glass, metal, paper, clothes, cardboard, biological waste, battery, and three other types of glass. Three different strategies were employed and tested: a baseline CNN trained from scratch, a deeper regularised CNN and a transfer learning model, pre-trained using MobileNetV2 on the ImageNet dataset. Even a relatively simple baseline CNN was able to achieve competitive accuracy of 75.39% on the validation set when trained for a sufficiently long period of time. CNN, though having more representational power, made it to 61.61% with the Adam optimiser and 59.06% with SGD+NM. At 96×96 resolution, an ablation study showed that the aggressive Dropout rates selected for the deeper model caused the model to diverge on this dataset. The best model was the MobileNetV2 fine tuned in ImageNet weights with a validation accuracy of 93.35%. The findings support the use of transfer learning as the most appropriate approach for: (1) real-world waste classification systems with limited training data and (2) systems with limited computational resources.

Table of Contents

1. Introduction
 2. Dataset Overview
 - 2.1 Class Distribution and Imbalance
 - 2.2 Data Split and Preprocessing
 - 2.3 Data Augmentation
 3. Methodology
 - 3.1 Baseline CNN Architecture
 - 3.2 Deeper CNN with Regularisation
 4. Experiments and Results
 - 4.1 Baseline vs. Deeper CNN
 - 4.2 Optimiser Comparison: Adam vs. SGD
 - 4.3 Ablation Study – Effect of Dropout
 5. Transfer Learning with MobileNetV2
 6. Overall Model Comparison
 7. Key Findings and Discussion
 8. Computational Efficiency
 9. Conclusion
 10. Future Work
- References

1. Introduction

Proper waste management stands out among some of the major environmental issues facing our world today. The process of urbanization, an increase in population, as well as consumption of nonreusable products have put tremendous strain on available garbage-processing facilities. Recyclable garbage often goes into landfills simply due to the fact that it has been sorted improperly at the point of dumping. The process of automated garbage sorting through deep learning offers an excellent solution to this issue due to its scalability and affordability.

Convolutional Neural Networks (CNNs) have become the dominant approach for image classification tasks, following the landmark work of (Alex Krizhevsky, 2012) on deep CNNs for large-scale image recognition. Their ability to automatically learn hierarchical visual features — from low-level edges and textures to high-level object structures — without manual feature engineering makes them well suited to the visual diversity found in garbage imagery. More recently, transfer learning techniques using models pre-trained on large datasets such as ImageNet have further improved classification accuracy while substantially reducing the volume of labelled training data required.

This report documents the design, training, and systematic evaluation of multiple deep learning architectures for classifying garbage images into 12 waste categories. The objectives of the project are:

- To build a baseline CNN model from scratch to serve as a reference architecture.
- To design a deeper, regularised CNN and evaluate its performance against the baseline.
- To compare the Adam and SGD optimisers in terms of convergence speed and classification accuracy.
- To isolate the effect of Dropout regularisation through a controlled ablation study.
- To apply transfer learning using MobileNetV2 and evaluate its performance advantage.
- To compare all models in terms of accuracy, training time, and practical deployment suitability.

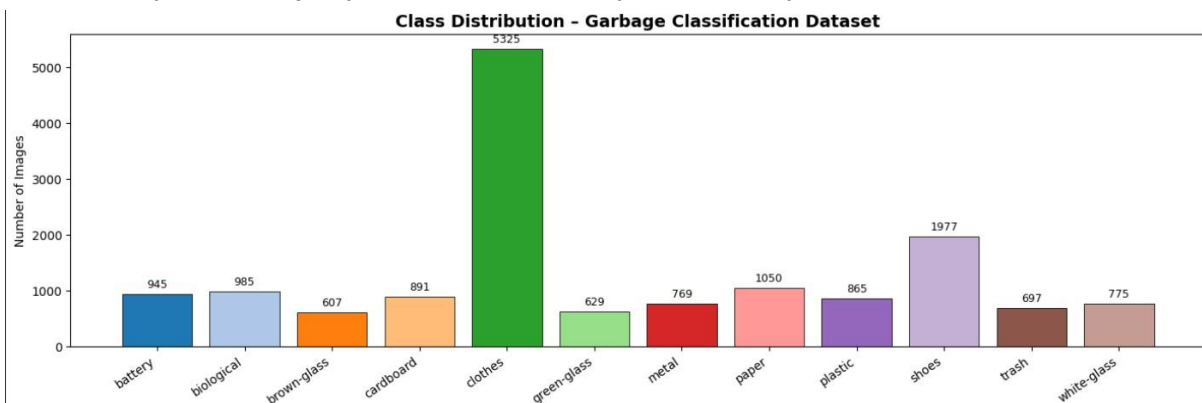
All models were trained on CPU hardware, which introduced meaningful computational constraints that influenced architectural choices and training configurations throughout the project. These constraints are discussed alongside the experimental results.

2. Dataset Overview

The project uses the Garbage Classification Dataset obtained from Kaggle (Mostafaabla, n.d.). The data consists of 15,515 images across 12 different garbage classes: battery, biological, brown glass, cardboard, clothes, green glass, metal, paper, plastic, shoes, trash, and white glass. In each image the background is relatively clean, and only one object is present, so the primary challenge is with the classification of material type and object shape, and not with the detection of objects in complex scenes.

2.1 Class Distribution and Imbalance

The data set is highly skewed. The classes vary significantly in size, with the largest class 'clothes' having 5325 images and the smallest 'brown-glass' having just 607 images, giving an imbalance ratio of around 8.8:1. The imbalance is great enough to give models a tendency to overestimate the accuracy of the majority classes, unless they are explicitly corrected.



Figure

1: Class distribution across the 12 garbage categories, showing significant imbalance (clothes: 5,325 images; brownglass: 607 images).

To address this, balanced class weights were computed using the `compute_class_weight('balanced')` function from Scikit-learn. The resulting weights were passed to all model training runs via the `class_weight` parameter of `model.fit()`. This ensures that minority classes exert proportionally greater influence on the loss function during training, improving recall on underrepresented categories such as brown-glass and biological waste.

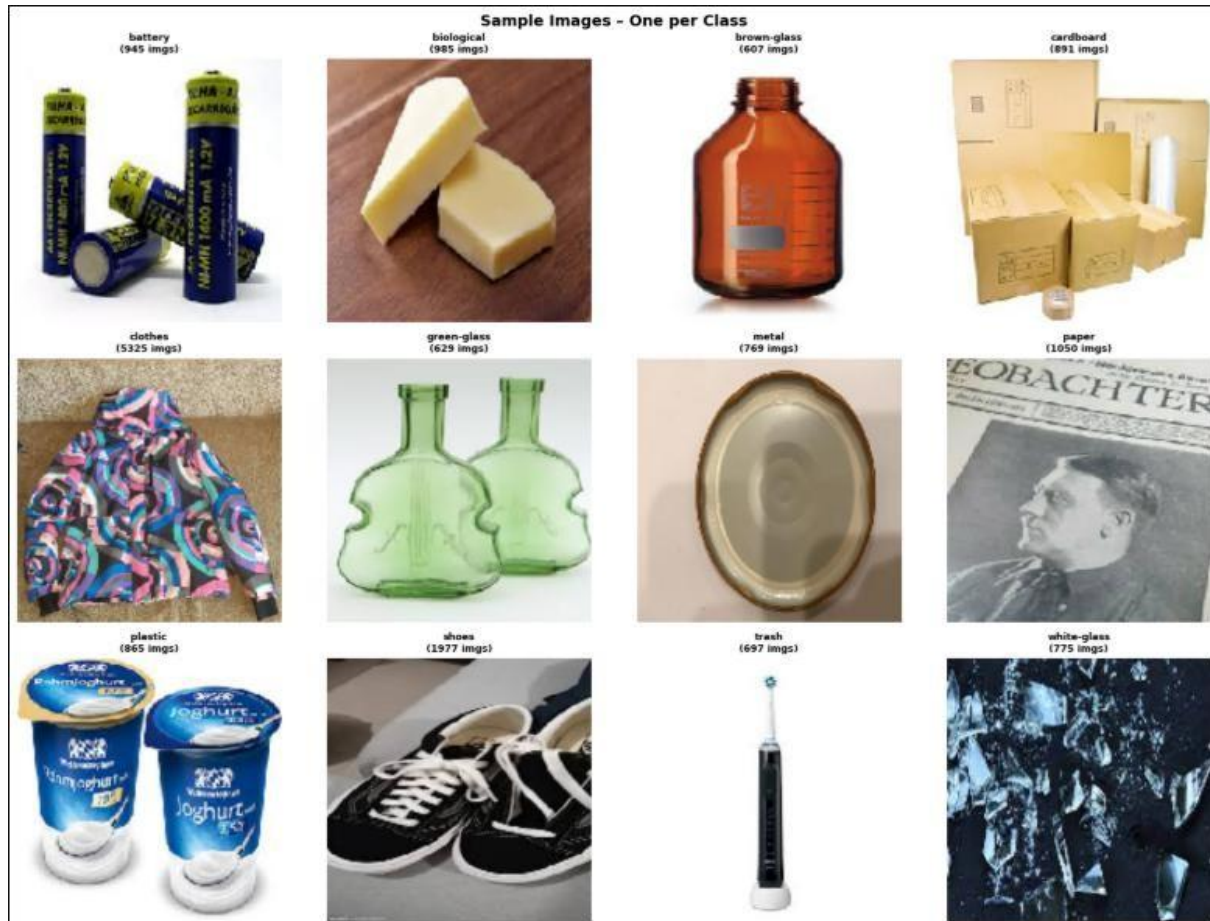


Figure 2: Representative sample images from each of the 12 waste categories with image counts shown.

2.2 Data Split and Preprocessing

An 80/20 train-validation split was applied using Keras's `ImageDataGenerator` with `validation_split=0.20` (Team, 2024). Two different image resolutions were used across the project. For the CNN models trained from scratch in Part A, images were resized to 96×96 pixels to reduce computational cost and make CPU training feasible within reasonable timeframes. For the transfer learning model in Part B, images were resized to 224×224 pixels to satisfy MobileNetV2's input requirements (Mark Sandler, 2018). In all cases, pixel values were normalised to the range [0, 1] by dividing by 255.

2.3 Data Augmentation

Data augmentation was applied exclusively to the training set to prevent data leakage into the validation set. The augmentation pipeline, implemented using TensorFlow's `ImageDataGenerator` (Team, 2024), included random rotations up to 20 degrees, horizontal and vertical shifts of up to 15%, a zoom range of 15%, horizontal flipping, and nearest-neighbour fill for border regions. Augmentation artificially increases the effective diversity of the training data by presenting the model with geometrically varied versions of each image, improving generalisation to unseen samples.

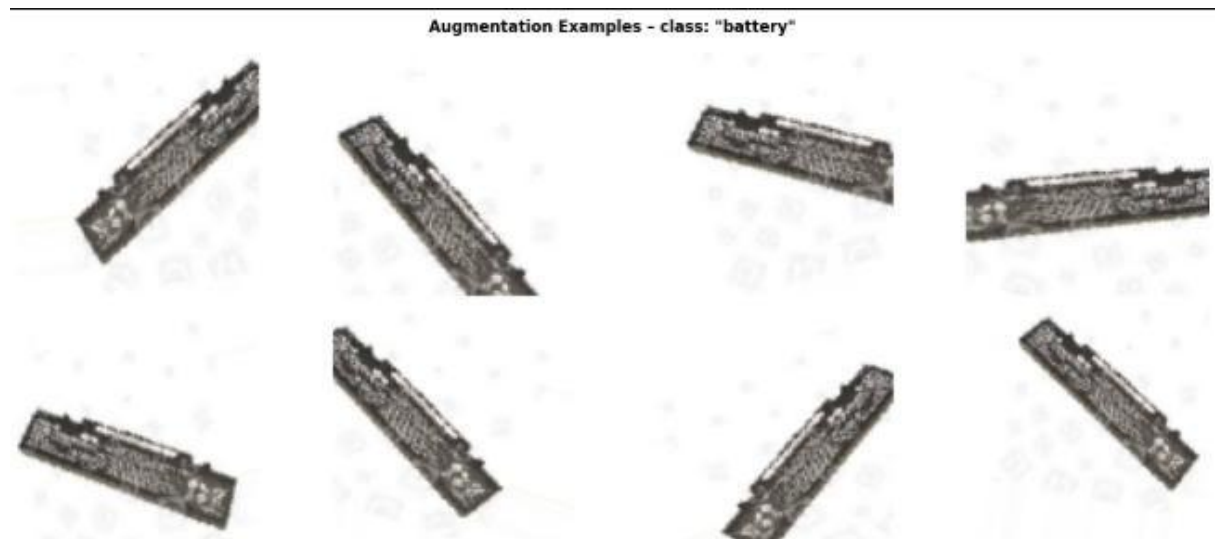


Figure 3: Examples of augmented images from the battery class, demonstrating the rotation, zoom, and flip variations applied during training.

3. Methodology

3.1 Baseline CNN Architecture

The baseline CNN was designed as a deliberately simple architecture, intentionally free of optimization techniques. Its purpose was to establish a well-understood reference point against which more complex models could be fairly compared. The architecture consists of three convolutional blocks, each containing a single Conv2D layer with 3×3 kernels and ReLU activations, followed by a 2×2 MaxPooling layer. This progression allows the network to learn increasingly abstract feature representations — from basic edges and corners in the first block to complex structural patterns and material textures in the third, a hierarchical learning behaviour first demonstrated at scale by [\(Alex Krizhevsky, 2012\)](#).

After the convolutional blocks, the feature maps are flattened and passed through three fully connected dense layers containing 256, 128, and 64 neurons respectively, before reaching the final 12-class Softmax output layer. The relatively small dense layer sizes were chosen to reduce memory consumption and training time given the 96×96 input resolution. No Batch Normalisation or Dropout layers were included in the baseline, making augmentation the sole mechanism for preventing overfitting. The model was compiled with the Adam Optimizer at a learning rate of 0.001 (Diederik P. Kingma, 2015) and categorical cross-entropy loss. EarlyStopping (patience = 5) and ReduceLROnPlateau (factor = 0.5, patience = 3) callbacks were applied to manage training duration and adapt the learning rate when validation performance plateaued.

```

Model: "Baseline_CNN"

```

Layer (type)	Output Shape	Param #
conv1 (Conv2D)	(None, 96, 96, 32)	896
pool1 (MaxPooling2D)	(None, 48, 48, 32)	0
conv2 (Conv2D)	(None, 48, 48, 64)	18,496
pool2 (MaxPooling2D)	(None, 24, 24, 64)	0
conv3 (Conv2D)	(None, 24, 24, 128)	73,856
pool3 (MaxPooling2D)	(None, 12, 12, 128)	0
flatten (Flatten)	(None, 18432)	0
fc1 (Dense)	(None, 256)	4,718,848
fc2 (Dense)	(None, 128)	32,896
fc3 (Dense)	(None, 64)	8,256
output (Dense)	(None, 12)	780

```

Total params: 4,854,028 (18.52 MB)

Trainable params: 4,854,028 (18.52 MB)

Non-trainable params: 0 (0.00 B)

```

Figure 4: Baseline CNN architecture summary showing layer types, output shapes, and total parameter count (4,854,028 trainable parameters).

3.2 Deeper CNN with Regularisation

The deeper CNN was developed to explore whether increasing architectural depth and adding explicit regularisation could improve upon the baseline. Unlike the baseline, each of the three convolutional blocks in this architecture contains two Conv2D layers rather than one, effectively doubling the depth of the feature extraction pathway. Each block follows the sequence: Conv2D → Batch Normalisation → ReLU activation → Conv2D → Batch Normalisation → ReLU → MaxPooling → Dropout. The filter sizes per block remain 32, 64, and 128 to maintain comparability with the baseline.

Batch Normalisation (Sergey Ioffe, 2015) was introduced to stabilise the internal activations between layers, reducing internal covariate shift and allowing smoother training at the specified learning rate. Dropout layers (Nitish Srivastava, 2014) were inserted after each pooling layer (rate = 0.25) and within the dense head (rates of 0.5 and 0.4) to combat overfitting by randomly deactivating neurons during training. L2 regularisation (coefficient = $1e-4$) was applied to the first two dense layers to penalise excessively large weights. The fully connected head used $512 \rightarrow 256 \rightarrow 128$ neurons, larger than the baseline to give the model sufficient capacity to combine the richer feature representations extracted by the deeper convolutional pathway. This model was trained using both Adam (Diederik P. Kingma, 2015) and SGD optimisers, as described in Section 4.2.

Model: "Deeper_CNN"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 96, 96, 32)	896
batch_normalization (BatchNormalization)	(None, 96, 96, 32)	128
activation (Activation)	(None, 96, 96, 32)	0
conv2d_1 (Conv2D)	(None, 96, 96, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 96, 96, 32)	128
activation_1 (Activation)	(None, 96, 96, 32)	0
max_pooling2d (MaxPooling2D)	(None, 48, 48, 32)	0
dropout (Dropout)	(None, 48, 48, 32)	0
conv2d_2 (Conv2D)	(None, 48, 48, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 48, 48, 64)	256
activation_2 (Activation)	(None, 48, 48, 64)	0
conv2d_3 (Conv2D)	(None, 48, 48, 64)	36,928
batch_normalization_3 (BatchNormalization)	(None, 48, 48, 64)	256
activation_3 (Activation)	(None, 48, 48, 64)	0
max_pooling2d_1 (MaxPooling2D)	(None, 24, 24, 64)	0
dropout_1 (Dropout)	(None, 24, 24, 64)	0
conv2d_4 (Conv2D)	(None, 24, 24, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 24, 24, 128)	512
activation_4 (Activation)	(None, 24, 24, 128)	0
conv2d_5 (Conv2D)	(None, 24, 24, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 24, 24, 128)	512
activation_5 (Activation)	(None, 24, 24, 128)	0
max_pooling2d_2 (MaxPooling2D)	(None, 12, 12, 128)	0
dropout_2 (Dropout)	(None, 12, 12, 128)	0
flatten_1 (Flatten)	(None, 18432)	0
dense (Dense)	(None, 512)	9,437,696
dropout_3 (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
dropout_4 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 128)	32,896
dense_3 (Dense)	(None, 12)	1,548

Figure 5: Deeper CNN architecture summary showing the increased depth with Batch Normalisation and Dropout layers inserted after each pooling block.

4. Experiments and Results

4.1 Baseline vs. Deeper CNN

The baseline CNN achieved a validation accuracy of 75.39% after 15 epochs of training, taking approximately 46.8 minutes on CPU. Training accuracy at epoch 15 was 78.78%, indicating a mild overfitting gap of around 3.4 percentage points. Validation loss began to fluctuate after epoch 11, suggesting the onset of memorisation, but the ReduceLROnPlateau callback successfully reduced the learning rate at epoch 14, allowing validation accuracy to recover to its peak. Given its simplicity and lack of regularisation, this performance is competitive and establishes a strong reference baseline.

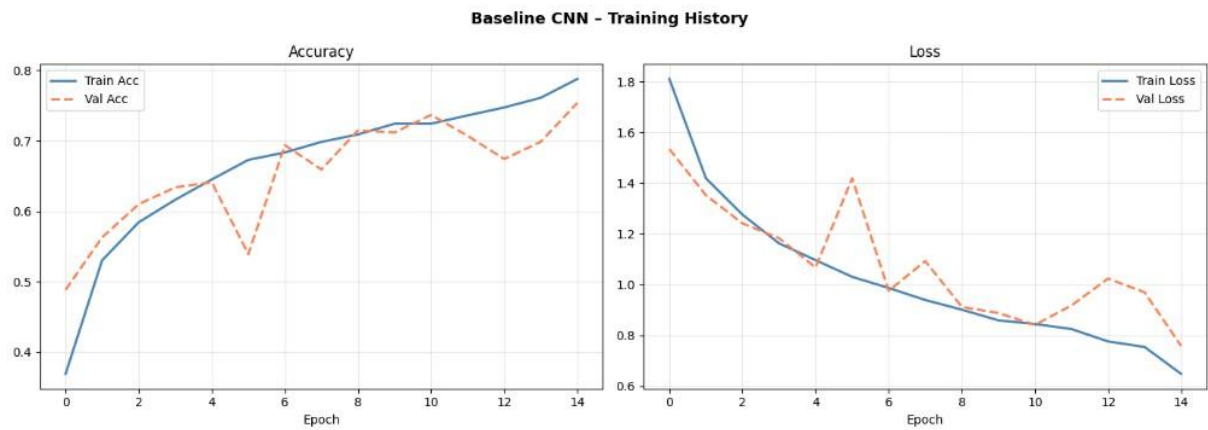


Figure 6: Baseline CNN training history showing training and validation accuracy/loss across 15 epochs. Validation loss fluctuation after epoch 11 signals the onset of mild overfitting.

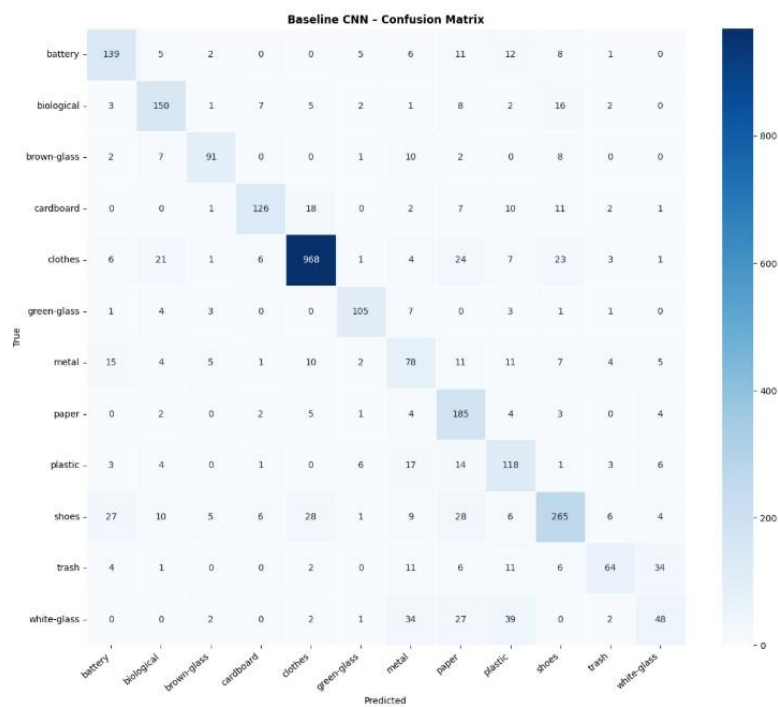


Figure 7: Baseline CNN confusion matrix across all 12 waste categories on the validation set. Strong diagonal performance is visible for classes such as clothes, shoes, and paper.

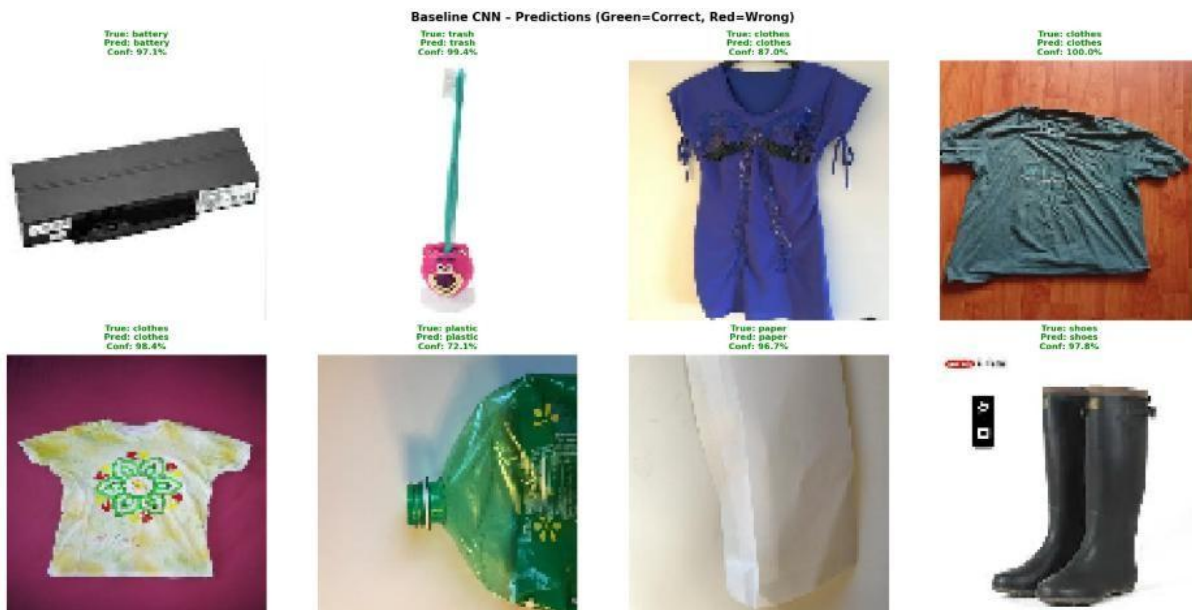


Figure 8: Sample predictions from the Baseline CNN. Green borders indicate correct classifications; red borders indicate errors.

The deeper CNN achieved 61.61% validation accuracy after 20 epochs, taking approximately 185.6 minutes on CPU — roughly four times longer than the baseline. This result is counterintuitive at first glance: a deeper, more regularized model performing worse than a simpler one. The explanation lies not in architectural deficiency but in the constraints imposed by CPU-only training. The combination of Batch Normalisation (Sergey Ioffe, 2015) and Dropout (Nitish Srivastava, 2014) significantly slows convergence; at epoch 20, training accuracy was only 59.72%, indicating the model had not yet finished learning the training data. Given GPU acceleration and 40–60 additional epochs, the deeper CNN would be expected to surpass the baseline due to its substantially greater representational capacity.

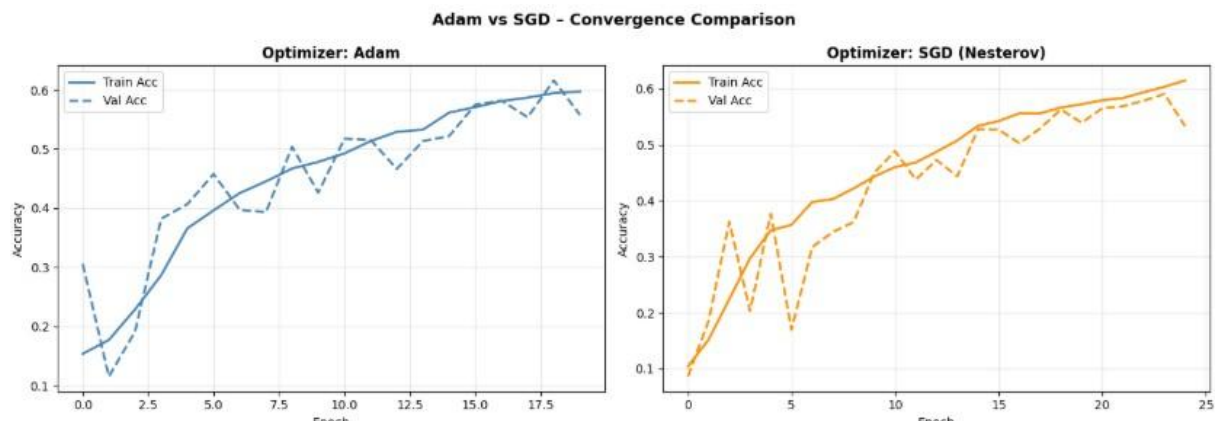


Figure 9: Deeper CNN (Adam) training history demonstrating the slow, gradual convergence caused by the combined overhead of Batch Normalisation and Dropout.

Model	Validation Accuracy	Training Time
Baseline CNN	75.39%	~46.8 min
Deeper CNN (Adam)	61.61%	~185.6 min

Table 1: Baseline CNN vs. Deeper CNN performance comparison.

4.2 Optimiser Comparison: Adam vs. SGD

The deeper CNN architecture was trained twice using different optimisers to evaluate their effect on convergence behaviour and final accuracy. Adam (Diederik P. Kingma, 2015) ($\text{lr} = 0.001$) was compared against SGD with Nesterov momentum ($\text{lr} = 0.01$, momentum = 0.9).

Optimiser	Validation Accuracy	Notes
Adam	61.61%	Faster convergence; adaptive per-parameter learning rate
SGD + Nesterov	59.06%	Slower convergence; required learning rate decay to progress

Table 2: Adam vs. SGD optimizer comparison on the deeper CNN.

Adam outperformed SGD by approximately 2.5 percentage points. As described by (Diederik P. Kingma, 2015), Adam's adaptive per-parameter learning rates allow it to navigate flat regions of the loss surface and saddle points more efficiently than SGD, making it particularly well suited to CPU constrained environments where training budgets are limited. SGD with Nesterov momentum can theoretically match or exceed Adam given sufficient training time and careful learning rate scheduling, but this advantage did not materialise within the available training budget. For practical purposes in this project, Adam is the clearly superior choice.

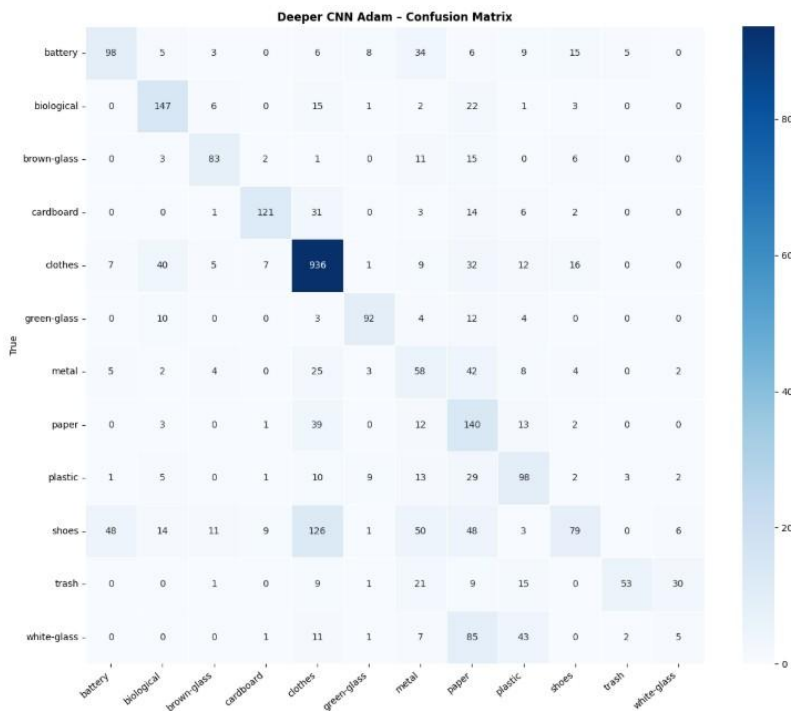


Figure 10: Deeper CNN (Adam) confusion matrix showing per-class classification performance. Notably weaker than the baseline, particularly on minority classes such as battery and trash.

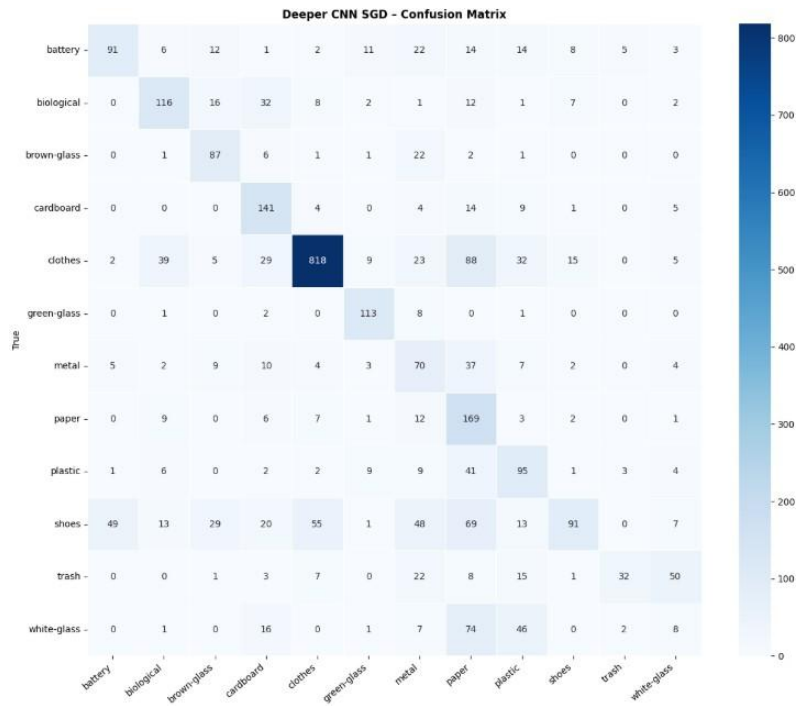


Figure 11: Deeper CNN (SGD + Nesterov) confusion matrix for direct comparison with the Adam variant. SGD achieves slightly lower accuracy with more diffuse off-diagonal errors.

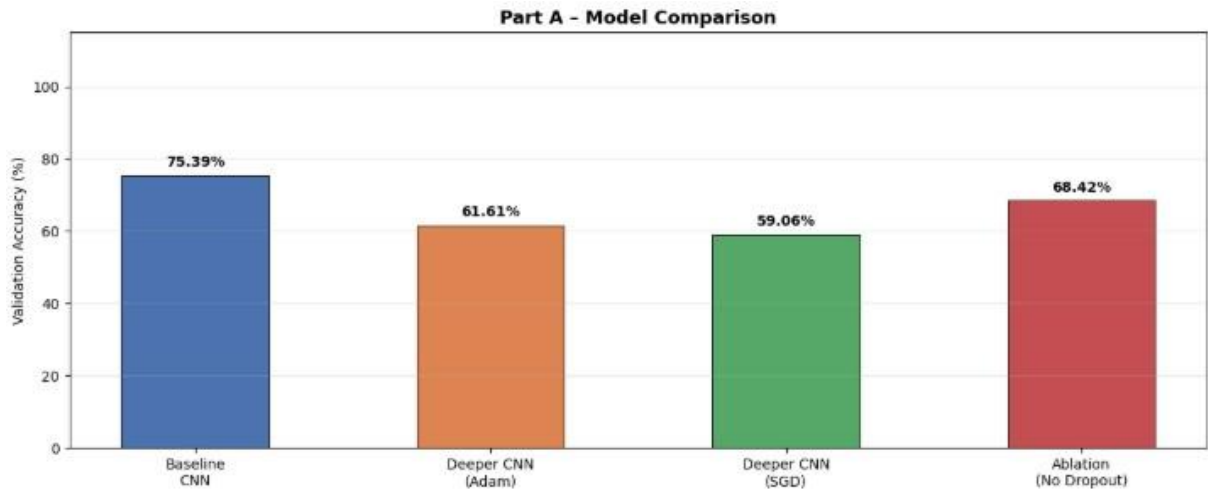


Figure 12: Validation accuracy comparison across all Part A configurations: Baseline CNN (75.39%), Deeper CNN Adam (61.61%), Deeper CNN SGD (59.06%), and Ablation (68.42%).

4.3 Ablation Study – Effect of Dropout

An ablation experiment was conducted to isolate the contribution of Dropout regularisation (Nitish Srivastava, 2014) to model performance. The deeper CNN was retrained with all Dropout layers removed while keeping every other architectural choice — including L2 regularisation on the dense layers — identical

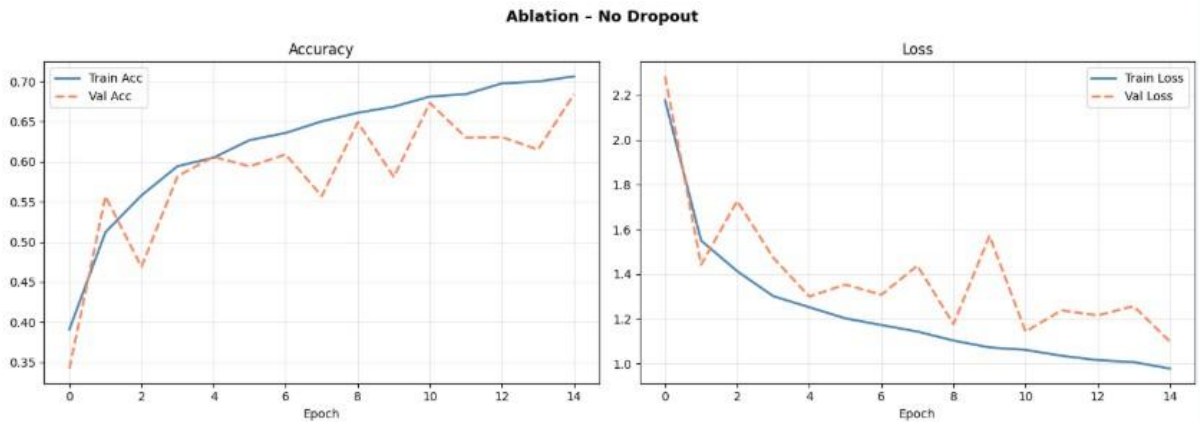


Figure 13: Ablation (no Dropout) training history showing faster and more stable convergence compared to the full Dropout configuration, confirming the Dropout rates were too aggressive.

Configuration	Validation Accuracy	Change
Deeper CNN with Dropout (0.25 / 0.5 / 0.4)	61.61%	—
Deeper CNN without Dropout (Ablation)	68.42%	+6.81%

Table 3: Effect of Dropout removal on validation accuracy.

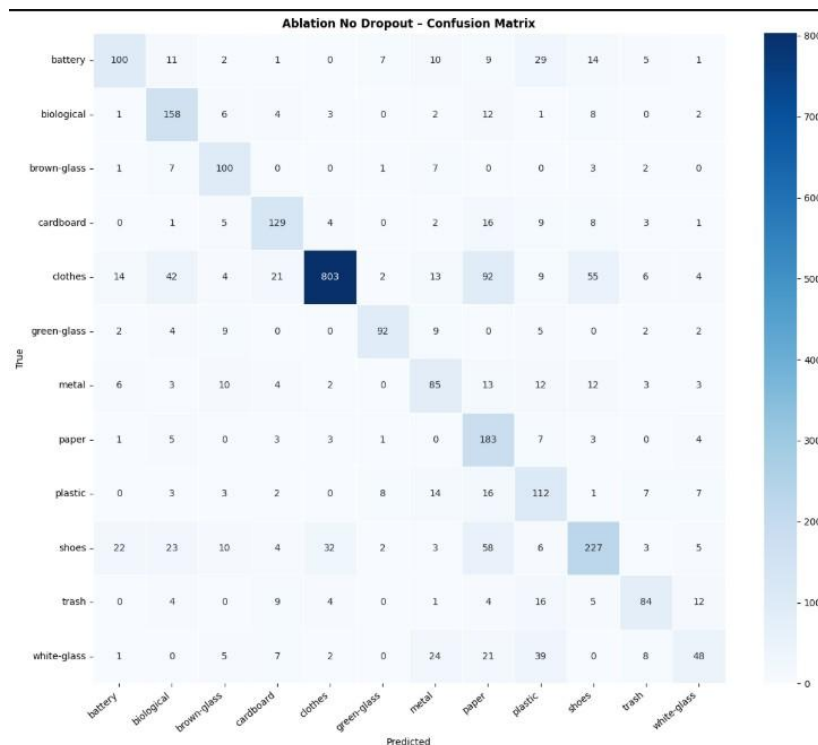


Figure 14: Ablation (no Dropout) confusion matrix showing improved per-class performance over the full Dropout variant, particularly on minority classes such as brown-glass and biological.

Removing Dropout improved validation accuracy by 6.81 percentage points. This finding indicates that the chosen Dropout rates were too aggressive for this specific combination of dataset size, image resolution, and available training time. At 96×96 resolution, the spatial feature maps are

relatively small and carry less redundant information than those produced at 224×224 . Suppressing 25–50% of activations at this resolution impedes convergence more than it prevents overfitting, particularly when training time is limited. Notably, the L2 regularisation alone was sufficient to provide meaningful overfitting protection in the ablation variant. A lighter Dropout configuration — such as rates of 0.1–0.2 per convolutional block and 0.3 in the dense head — would likely produce a better balance between regularisation and convergence speed.

5. Transfer Learning with MobileNetV2

Transfer learning was implemented using MobileNetV2 (Mark Sandler, 2018), a lightweight depthwise-separable convolutional architecture pre-trained on ImageNet (1.2 million images, 1,000 classes) (Alex Krizhevsky, 2012). MobileNetV2 encodes a rich hierarchy of visual features: low-level filters detect edges and gradients, mid-level filters capture textures and surface patterns, and higherlevel filters represent object shapes and semantic parts. These representations are directly relevant to garbage classification, where distinguishing material types such as glass, cardboard, metal, and plastic relies on exactly these visual cues.

The classification head of MobileNetV2 was replaced with a custom dense stack ($512 \rightarrow 256 \rightarrow 12$ neurons with Softmax output) to adapt the model to the 12-class garbage task. A two-phase training strategy was employed:

Phase 1 – Feature Extraction: The entire MobileNetV2 base was frozen and only the new classification head was trained for up to 12 epochs. This stabilised the head weights before any modification to the pre-trained convolutional layers, preventing unstable gradients from corrupting the ImageNet representations (Andrew G. Howard, 2017).

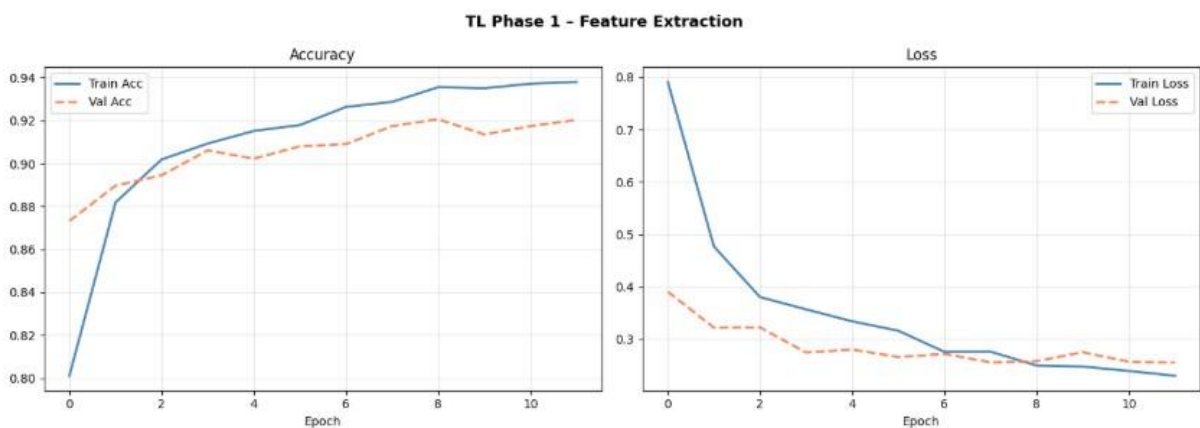
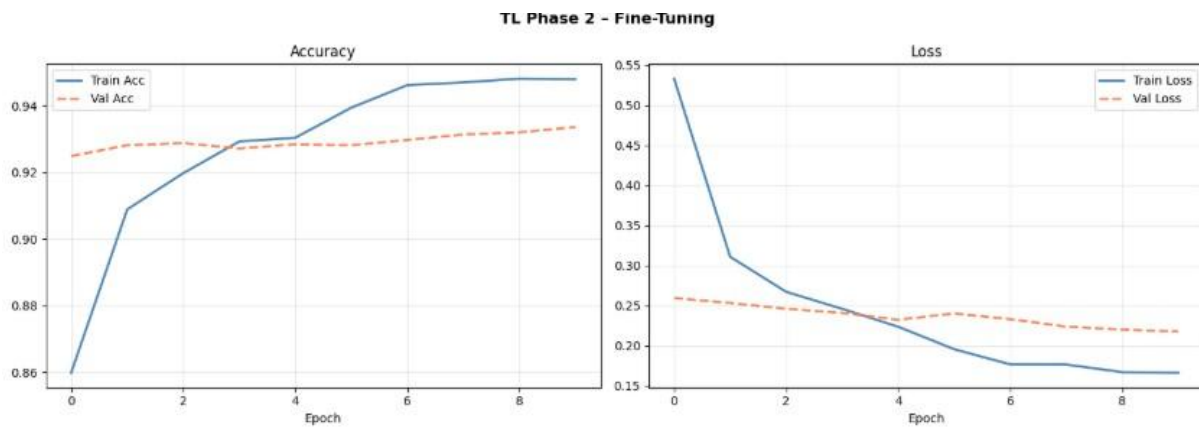


Figure 15: MobileNetV2 Phase 1 (feature extraction) training history. The frozen base allows the classification head to reach high accuracy rapidly within 10 epochs.

Phase 2 – Fine-Tuning: The top 30 layers of MobileNetV2 were unfrozen and trained at a very low learning rate ($lr = 1e-5$) using the Adam optimizer (Diederik P. Kingma, 2015) for up to 10 additional epochs. This allowed the upper convolutional layers to adapt to the specific visual properties of garbage images while the very low learning rate prevented catastrophic forgetting of the lower-level ImageNet features.



Figure

16: MobileNetV2 Phase 2 (fine-tuning) training history. Unfreezing the top 30 layers at $lr = 1e-5$ yields further accuracy gains without disrupting the pre-trained lower-level features.

Images were resized to 224×224 pixels for this phase, as required by MobileNetV2’s architecture (Mark Sandler, 2018), with the same augmentation pipeline used in Part A applied to the training set (Team, 2024).

MobileNetV2 achieved a validation accuracy of 93.35% — outperforming the best from-scratch result (75.39%) by nearly 18 percentage points. Phase 1 alone produced a competitive baseline, confirming that ImageNet feature representations transfer well to garbage classification. Phase 2 fine-tuning provided additional gains by adapting the higher-level convolutional layers to the specific textures and shapes characteristic of the waste categories in this dataset.

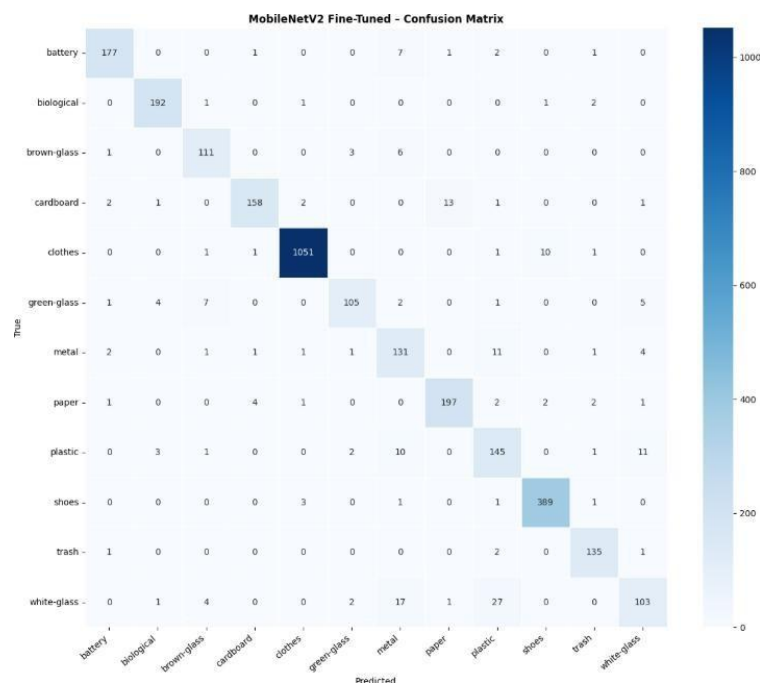


Figure 17: MobileNetV2 fine-tuned confusion matrix demonstrating near-diagonal performance across all 12 classes. Residual errors are concentrated in visually similar categories such as white-glass and green-glass.

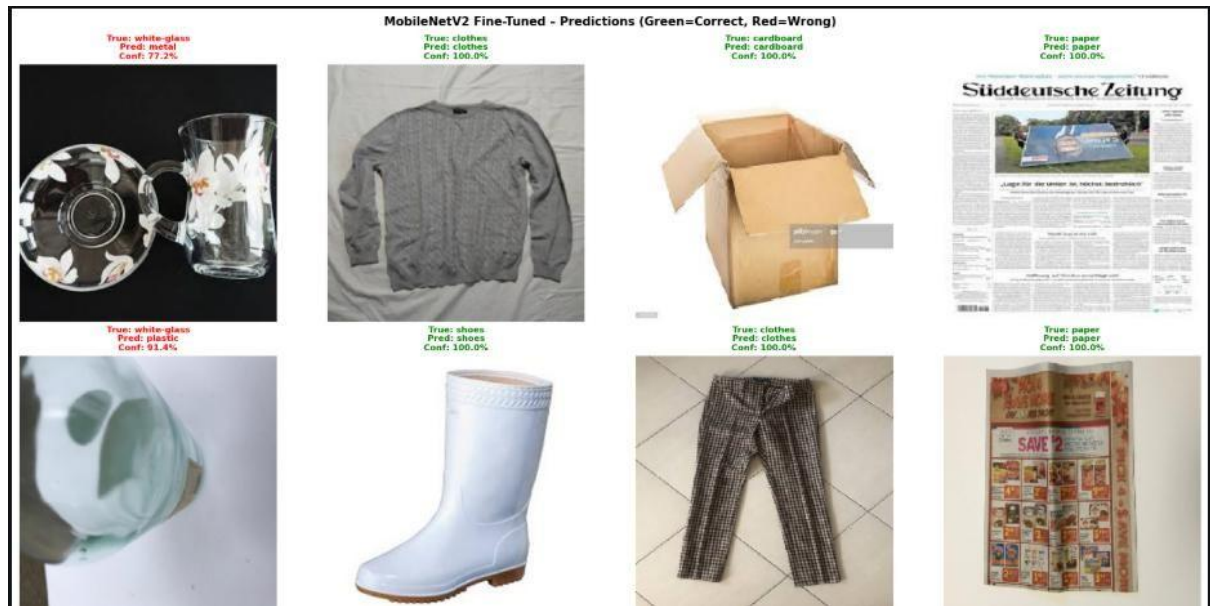


Figure 18: Sample MobileNetV2 predictions showing high-confidence correct classifications across diverse waste types including clothes, cardboard, shoes, and paper.

The result also demonstrates that with only approximately 1,250 images per class on average, transfer learning is far more data-efficient than training a comparably deep network from random initialization, which would risk severe overfitting at this data scale.

6. Overall Model Comparison

Model	Val. Accuracy	Approx. Train Time	Regularisation
Baseline CNN	75.39%	~46.8 min	None (augmentation only)
Deeper CNN (Adam)	61.61%	~185.6 min	BatchNorm + Dropout + L2
Deeper CNN (SGD)	59.06%	~185+ min	BatchNorm + Dropout + L2
Ablation (No Dropout)	68.42%	~120 min	L2 only
MobileNetV2 Fine-Tuned	93.35%	~110 min	Dropout in head, frozen base

Table 4: Summary of all model results.

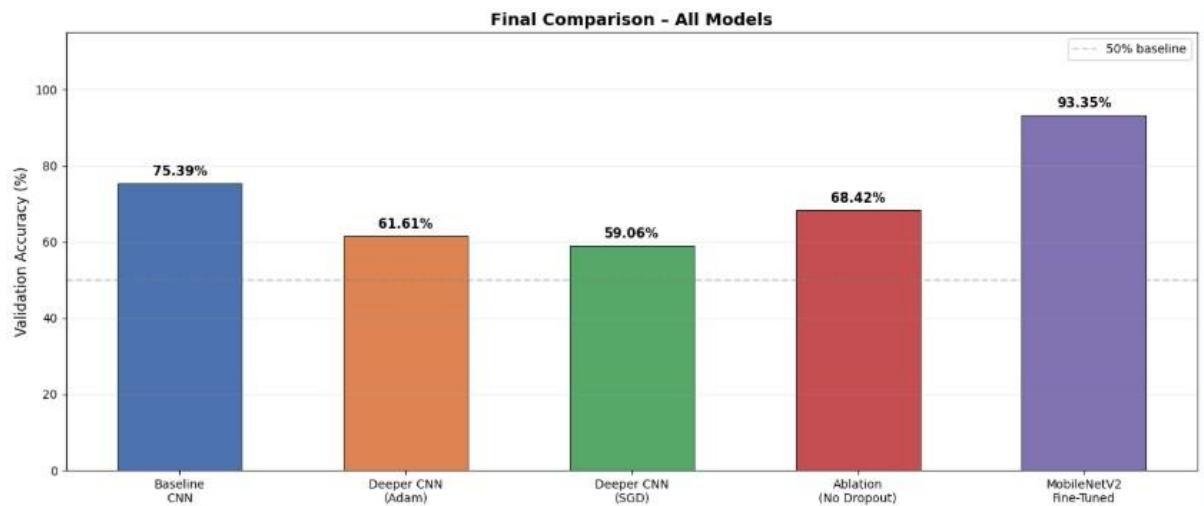


Figure 19: Final validation accuracy comparison across all five model configurations. MobileNetV2 fine-tuned achieves 93.35%, far exceeding all from-scratch models. The 50% random-chance baseline is shown as a dashed reference line.

MobileNetV2 achieves the highest validation accuracy at 93.35%, outperforming every from-scratch architecture by a substantial margin. The baseline CNN is surprisingly competitive given its simplicity, outperforming the deeper regularized variants primarily because CPU training constraints prevented the deeper model from converging fully. The ablation experiment confirms that Dropout rates require careful calibration to avoid degrading performance on lower-resolution inputs.

7. Key Findings and Discussion

7.1 Architectural Depth Does Not Guarantee Better Performance

The deeper CNN's underperformance relative to the baseline is one of the most instructive results of this project. Adding more layers and regularisation does not automatically improve accuracy — it also increases the number of epochs required for convergence. The Batch Normalisation layers (Sergey Ioffe, 2015) and dual-convolution blocks increased training time fourfold without yielding proportional accuracy gains under CPU constraints. Given GPU acceleration and 60+ epochs, the deeper architecture's greater representational capacity would be expected to surpass the baseline.

7.2 Dropout Rates Must Be Calibrated to Input Resolution

The ablation study demonstrates a subtle but important finding: (Nitish Srivastava, 2014) Dropout rates that might be appropriate for larger input resolutions can be too aggressive at 96×96 , where feature maps are smaller and carry less spatial redundancy. Suppressing too many activations at this resolution slows convergence unnecessarily and effectively wastes training time. This suggests that regularisation configurations should not be transferred between architectures or input resolutions without re-tuning. A systematic search over Dropout rates and L2 coefficients would likely yield meaningful improvements for the deeper architecture.

7.3 Adam Is the Preferred Optimizer in Resource-Constrained Settings

As demonstrated empirically and described theoretically by (Diederik P. Kingma, 2015), Adam's adaptive per-parameter learning rates allow faster and more reliable convergence than SGD in environments where training time is limited. While well-tuned SGD with momentum can match or slightly exceed Adam given sufficient compute, the engineering overhead is rarely justified for practical projects of this scale.

7.4 Class Imbalance Correction Is Essential

Balanced class weights were a necessary component of the training pipeline. Without this correction, models would develop strong biases towards the majority class (clothes, 5,325 images) and largely ignore minority categories such as brown-glass (607 images). Although overall accuracy might appear reasonable in such a scenario, the model would fail on precisely the waste categories that require the most attention in real-world sorting systems. Balanced weighting ensures the model learns all 12 categories with comparable effectiveness.

7.5 Transfer Learning Is the Definitive Best Approach

MobileNetV2’s 93.35% validation accuracy demonstrates the effectiveness of transfer learning for medium-scale image classification tasks. This success is largely due to the strong alignment between ImageNet’s learned visual feature hierarchy and the texture and shape characteristics required for waste material classification. The lightweight architecture of MobileNetV2 (Andrew G. Howard, 2017; Mark Sandler, 2018) additionally makes it well suited for deployment on mobile and edge devices, an important practical consideration for real-world waste sorting systems.

8. Computational Efficiency

All models in this project were trained on CPU hardware, which imposed significant time constraints on the experiments. The table below summarises approximate training times for each model.

Model	Approx. Training Time	Notes
Baseline CNN	~46.8 min	Fastest; simple architecture, 96×96 input
Deeper CNN (Adam/SGD)	~185.6 min	4× slower; BatchNorm and dual conv layers add cost
Ablation (No Dropout)	~120 min	Faster convergence without Dropout overhead
MobileNetV2 (both phases)	~110 min	Higher per-epoch cost at 224×224 offset by fewer epochs needed

Table 5: Approximate training times across all models.

The deeper CNN required approximately four times longer to train than the baseline, largely due to the additional convolutional layers, Batch Normalisation operations, and the inherently slower convergence induced by heavy Dropout. MobileNetV2, despite requiring larger 224×224 inputs, benefited from the efficiency of its depthwise-separable convolutions and the fact that only the classification head was trained during Phase 1. The use of GPU or TPU hardware via platforms such as Google Colab would reduce training times for all models by an order of magnitude, enabling more extensive hyperparameter tuning and longer training runs that would likely close the performance gap between the deeper CNN and the baseline.

9. Conclusion

This project systematically investigated and compared multiple deep learning approaches for automated 12-class garbage image classification. Beginning with a simple baseline CNN and

progressing through a deeper regularised architecture and a transfer learning model, the results clearly demonstrate the superiority of transfer learning for this class of problem.

The baseline CNN achieved 75.39% validation accuracy, confirming that simple architectures trained from scratch can perform competitively when computational resources are limited. The deeper CNN, despite its greater theoretical capacity, achieved only 61.61% due to incomplete convergence under CPU time constraints — an instructive reminder that architectural complexity must be matched to available hardware. The ablation study revealed that the chosen Dropout rates were too aggressive at 96×96 resolution, providing a clear direction for future regularisation tuning. The optimiser comparison confirmed Adam's practical advantage over SGD in resource-constrained settings.

MobileNetV2, fine-tuned using a two-phase transfer learning strategy, achieved 93.35% validation accuracy — nearly 18 percentage points above the best from-scratch result. This result firmly establishes transfer learning as the recommended approach for garbage classification systems, particularly in scenarios where training data per class is limited and deployment on mobile or edge hardware is required.

10. Future Work

Several directions could meaningfully extend and improve upon the results presented in this report:

- GPU/TPU training: Retraining the deeper CNN on GPU hardware for 60+ epochs would provide a fairer evaluation of its true performance ceiling and allow a more definitive comparison with the baseline.
- Dropout rate tuning: A systematic search over Dropout rates (e.g., 0.1, 0.15, 0.2 per convolutional block) at 96×96 resolution would likely yield a deeper CNN configuration that outperforms both the current deeper model and the ablation variant.
- Alternative backbones: EfficientNetB0 and ResNet50 offer different accuracy-efficiency trade-offs and would be worth benchmarking against MobileNetV2 on this dataset.
- Learning rate scheduling: Cosine annealing or warm-restart schedules for SGD could close the performance gap between SGD and Adam observed in this project.
- Dataset expansion: Collecting additional images for minority classes such as brown-glass and biological waste would reduce the imbalance ratio and likely improve classification accuracy on these difficult categories.
- Real-time deployment: Quantisation-aware training and model pruning could reduce MobileNetV2's memory footprint further, enabling deployment on embedded devices and mobile applications for real-time waste sorting.
- Test set evaluation: Adding a held-out test set separate from the validation split would provide a more reliable and unbiased estimate of generalisation performance.

References

- Alex Krizhevsky, I. S. G. E. H., 2012. *ImageNet Classification with Deep Convolutional Neural Networks*. [Online] Available at: https://papers.nips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf
- Andrew G. Howard, M. Z. B. C. D. K. W. W. T. W. M. A. H. A., 2017. *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. [Online] Available at: <https://arxiv.org/abs/1704.04861>
- Diederik P. Kingma, J. B., 2015. *Adam: A Method for Stochastic Optimization*. [Online] Available at: <https://arxiv.org/abs/1412.6980>

Mark Sandler, A. H. M. Z. A. Z. L.-C. C., 2018. *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. [Online]

Available at: <https://arxiv.org/abs/1801.04381>

Mostafaabla, n.d. *Garbage Classification*. [Online]

Available at: <https://www.kaggle.com/datasets/mostafaabla/garbage-classification/data> [Accessed 2023].

Nitish Srivastava, G. H. A. K. I. S. R. S., 2014. *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*. [Online]

Available at: <https://jmlr.org/papers/v15/srivastava14a.html>

Sergey Ioffe, C. S., 2015. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. [Online]

Available at: <https://arxiv.org/abs/1502.03167>

Team, F. C. /. K., 2024. *Image data preprocessing*. [Online]

Available at: https://keras.io/api/data_loading/image/

GITHUB: https://github.com/sansar-dngol/AI_ML_Workshop.git